

TP n°4 — Introduction au PL/SQL Oracle

Introduction

Le PL/SQL (Procedural Language/SQL) est une extension procédurale du langage SQL propre à Oracle. Il permet d'écrire des blocs de code combinant instructions SQL et structures de contrôle procédurales (conditions, boucles, curseurs, gestion d'exceptions, packages). Ce TP en introduit les fondamentaux à travers trois exercices progressifs.

Prérequis : activer l'affichage des sorties avant toute exécution.

```
SET SERVEROUTPUT ON;
```

Exercice 1 : Blocs anonymes et fonctions

Question 1 — Somme de deux entiers

Ce bloc anonyme invite l'utilisateur à saisir deux entiers via des variables de substitution et affiche leur somme.

```
SET SERVEROUTPUT ON;
DECLARE
    v_a      NUMBER := &premier_entier;
    v_b      NUMBER := &deuxieme_entier;
    v_somme  NUMBER;
BEGIN
    v_somme := v_a + v_b;
    DBMS_OUTPUT.PUT_LINE('Somme : ' || v_somme);
END;
/
```

Résultat attendu (ex. a=5, b=3) :

```
Somme : 8
```

*Note : Les variables de substitution &nom sont remplacées par la valeur saisie au moment de l'exécution dans SQL*Plus ou SQL Developer.*

Question 2 — Table de multiplication

Ce bloc affiche la table de multiplication (de 1 à 10) d'un entier saisi par l'utilisateur, en utilisant une boucle FOR.

```
SET SERVEROUTPUT ON;
DECLARE
    v_n NUMBER := &nombre;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Table de multiplication de ' || v_n || ' :');
    FOR v_i IN 1..10 LOOP
        DBMS_OUTPUT.PUT_LINE(v_n || ' x ' || v_i || ' = ' || (v_n * v_i));
    END LOOP;
END;
/
```

Résultat attendu (ex. n=3) :

```
Table de multiplication de 3 :
3 x 1 = 3
3 x 2 = 6
...
3 x 10 = 30
```

Question 3 — Fonction récursive : x puissance n

Cette fonction récursive retourne x^n pour x et n entiers positifs. Le cas de base est n=0 (retourne 1), le cas récursif multiplie x par puissance(x, n-1).

```
CREATE OR REPLACE FUNCTION puissance(p_x IN NUMBER, p_n IN NUMBER)
RETURN NUMBER IS
BEGIN
    IF p_n = 0 THEN
        RETURN 1;
    ELSIF p_n < 0 THEN
        RETURN NULL;
    ELSE
        RETURN p_x * puissance(p_x, p_n - 1);
    END IF;
END puissance;
/

-- Test
BEGIN
    DBMS_OUTPUT.PUT_LINE('2^10 = ' || puissance(2, 10));
    DBMS_OUTPUT.PUT_LINE('3^4 = ' || puissance(3, 4));
    DBMS_OUTPUT.PUT_LINE('5^0 = ' || puissance(5, 0));
END;
/
```

Résultats attendus :

```
2^10 = 1024
3^4 = 81
5^0 = 1
```

Note : La récursivité en PL/SQL fonctionne comme dans tout langage procédural. Il faut toujours définir un cas de base pour éviter une récursion infinie.

Question 4 — Factorielle et stockage dans une table

Ce bloc calcule la factorielle d'un entier strictement positif saisi par l'utilisateur et stocke le résultat dans la table resultatFactoriel.

```
-- Création de la table de stockage
CREATE TABLE resultatFactoriel (
    nombre          NUMBER(10),
    factorielle      NUMBER(38)
);

-- Bloc de calcul
SET SERVEROUTPUT ON;
DECLARE
    v_n      NUMBER := &nombre_positif;
    v_fact   NUMBER := 1;
BEGIN
    IF v_n <= 0 THEN
        DBMS_OUTPUT.PUT_LINE('Erreur : le nombre doit être strictement positif.');
```

```
    ELSE
        FOR v_i IN 1..v_n LOOP
            v_fact := v_fact * v_i;
        END LOOP;
        INSERT INTO resultatFactoriel VALUES (v_n, v_fact);
```

```

        COMMIT;
        DBMS_OUTPUT.PUT_LINE(v_n || ' ! = ' || v_fact);
    END IF;
END;
/

```

Résultat attendu (ex. n=5) :

```
5! = 120
```

Question 5 — Factorielles des 20 premiers entiers

Extension du bloc précédent : calcul et stockage des factorielles de 1 à 20 en une seule exécution. On réutilise la valeur précédente à chaque itération pour optimiser le calcul.

```

CREATE TABLE resultatsFactoriels (
    nombre          NUMBER(10),
    factorielle      NUMBER(38)
);

SET SERVEROUTPUT ON;
DECLARE
    v_fact NUMBER := 1;
BEGIN
    FOR v_n IN 1..20 LOOP
        v_fact := v_fact * v_n;
        INSERT INTO resultatsFactoriels VALUES (v_n, v_fact);
        DBMS_OUTPUT.PUT_LINE(v_n || ' ! = ' || v_fact);
    END LOOP;
    COMMIT;
END;
/

```

Résultats attendus (extrait) :

```

1! = 1
2! = 2
5! = 120
10! = 3628800
20! = 2432902008176640000

```

*Note : On calcule $v_fact := v_fact * v_n$ à chaque itération plutôt que de repartir de 1. Cela évite de recalculer toute la suite et optimise l'exécution.*

Exercice 2 : Curseurs et manipulation de la table EMP

La table emp est créée et alimentée avec des données de test. Tous les blocs suivants supposent cette initialisation effectuée.

```

CREATE TABLE emp (
    matr          NUMBER(10) NOT NULL,
    nom           VARCHAR2(50) NOT NULL,
    sal           NUMBER(7,2),
    adresse       VARCHAR2(96),
    dep           NUMBER(10) NOT NULL,
    CONSTRAINT emp_pk PRIMARY KEY (matr)
);

INSERT INTO emp VALUES (1, 'Alice', 3000, '10 rue de Paris', 92000);
INSERT INTO emp VALUES (2, 'Bob', 2500, '5 avenue Victor Hugo', 75000);
INSERT INTO emp VALUES (3, 'Carlos', 4000, '8 boulevard Voltaire', 92000);
INSERT INTO emp VALUES (4, 'Youcef', 2500, 'avenue Anatole France', 92000);
INSERT INTO emp VALUES (5, 'Diana', 3500, '22 rue Lafayette', 75000);
INSERT INTO emp VALUES (6, 'Emma', NULL, '3 place du Marché', 10);

```

```
COMMIT;
```

Question 1 — Insertion d'un employé via %ROWTYPE

Ce bloc utilise %ROWTYPE pour déclarer une variable dont la structure correspond exactement à une ligne de emp, puis insère l'employé.

```
SET SERVEROUTPUT ON;
DECLARE
    v_employe emp%ROWTYPE;
BEGIN
    v_employe.matr      := 4;
    v_employe.nom       := 'Youssef';
    v_employe.sal       := 2500;
    v_employe.adresse   := 'avenue Anatole France';
    v_employe.dep       := 92000;
    INSERT INTO emp VALUES v_employe;
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Employé inséré avec succès.');
```

```
END;
/
```

Note : %ROWTYPE évite de redéclarer chaque attribut individuellement. La structure de la variable s'adapte automatiquement à celle de la table.

Question 2 — Suppression avec SQL%ROWCOUNT

Ce bloc supprime tous les employés du département 10 et affiche le nombre de lignes supprimées grâce à l'attribut de curseur implicite SQL%ROWCOUNT.

```
SET SERVEROUTPUT ON;
DECLARE
    v_nb_lignes NUMBER;
BEGIN
    DELETE FROM emp WHERE dep = 10;
    v_nb_lignes := SQL%ROWCOUNT;
    DBMS_OUTPUT.PUT_LINE('Nombre de lignes supprimées : ' || v_nb_lignes);
    COMMIT;
END;
/
```

Résultat attendu (1 employé dans dep=10) :

```
Nombre de lignes supprimées : 1
```

Note : SQL%ROWCOUNT est un attribut du curseur implicite. Il retourne le nombre de lignes affectées par la dernière instruction DML (INSERT, UPDATE, DELETE).

Question 3 — Somme des salaires avec curseur explicite (LOOP)

Ce bloc utilise un curseur explicite et la boucle LOOP ... EXIT WHEN ... END LOOP pour calculer la somme totale des salaires, en ignorant les valeurs NULL.

```
SET SERVEROUTPUT ON;
DECLARE
    v_salaire EMP.sal%TYPE;
    v_total   EMP.sal%TYPE := 0;
    CURSOR c_salaires IS
        SELECT sal FROM emp;
BEGIN
    OPEN c_salaires;
    LOOP
        FETCH c_salaires INTO v_salaire;
        EXIT WHEN c_salaires%NOTFOUND;
        IF v_salaire IS NOT NULL THEN
            v_total := v_total + v_salaire;
```

```

        END IF;
    END LOOP;
    CLOSE c_salaires;
    DBMS_OUTPUT.PUT_LINE('Somme des salaires : ' || v_total);
END;
/

```

Résultat attendu (Alice+Bob+Carlos+Youssef+Diana = 3000+2500+4000+2500+3500) :

```
Somme des salaires : 15500
```

Note : Le curseur explicite suit 3 étapes : OPEN (ouverture), FETCH (lecture ligne par ligne), CLOSE (fermeture). %NOTFOUND devient TRUE quand il n'y a plus de lignes à lire.

Question 4 — Salaire moyen avec curseur explicite (LOOP)

Modification du bloc précédent pour calculer la moyenne. Un compteur v_count est ajouté pour compter les employés ayant un salaire non NULL.

```

SET SERVEROUTPUT ON;
DECLARE
    v_salaire EMP.sal%TYPE;
    v_total    EMP.sal%TYPE := 0;
    v_count    NUMBER       := 0;
    CURSOR c_salaires IS SELECT sal FROM emp;
BEGIN
    OPEN c_salaires;
    LOOP
        FETCH c_salaires INTO v_salaire;
        EXIT WHEN c_salaires%NOTFOUND;
        IF v_salaire IS NOT NULL THEN
            v_total := v_total + v_salaire;
            v_count := v_count + 1;
        END IF;
    END LOOP;
    CLOSE c_salaires;
    IF v_count > 0 THEN
        DBMS_OUTPUT.PUT_LINE('Salaire moyen : ' || ROUND(v_total / v_count,
2));
    ELSE
        DBMS_OUTPUT.PUT_LINE('Aucun salaire disponible.');

```

Résultat attendu :

```
Salaire moyen : 3100
```

Question 5 — Somme et moyenne avec la boucle FOR IN

Les deux procédures précédentes sont réécrites avec la boucle FOR IN, qui gère automatiquement OPEN, FETCH et CLOSE.

Somme des salaires :

```

SET SERVEROUTPUT ON;
DECLARE
    v_total EMP.sal%TYPE := 0;
    CURSOR c_salaires IS SELECT sal FROM emp;
BEGIN
    FOR v_emp IN c_salaires LOOP
        IF v_emp.sal IS NOT NULL THEN
            v_total := v_total + v_emp.sal;
        END IF;
    END LOOP;
    DBMS_OUTPUT.PUT_LINE('Somme des salaires : ' || v_total);

```

```
END;  
/
```

Salaire moyen :

```
SET SERVEROUTPUT ON;  
DECLARE  
    v_total EMP.sal%TYPE := 0;  
    v_count NUMBER       := 0;  
    CURSOR c_salaires IS SELECT sal FROM emp;  
BEGIN  
    FOR v_emp IN c_salaires LOOP  
        IF v_emp.sal IS NOT NULL THEN  
            v_total := v_total + v_emp.sal;  
            v_count := v_count + 1;  
        END IF;  
    END LOOP;  
    IF v_count > 0 THEN  
        DBMS_OUTPUT.PUT_LINE('Salaire moyen : ' || ROUND(v_total / v_count,  
2));  
    END IF;  
END;  
/
```

Note : La boucle FOR IN est plus concise. La variable de boucle (v_emp) est déclarée implicitement avec le type de la ligne retournée par le curseur. OPEN, FETCH et CLOSE sont gérés automatiquement.

Question 6 — Curseur paramétré par département

Ce bloc utilise un curseur paramétré pour afficher les noms des employés des départements 92000 et 75000. Le même curseur est réutilisé avec des paramètres différents.

```
SET SERVEROUTPUT ON;  
DECLARE  
    CURSOR c(p_dep EMP.dep%TYPE) IS  
        SELECT dep, nom  
        FROM emp  
        WHERE dep = p_dep;  
BEGIN  
    DBMS_OUTPUT.PUT_LINE('--- Département 92000 ---');  
    FOR v_employe IN c(92000) LOOP  
        DBMS_OUTPUT.PUT_LINE(' ' || v_employe.nom);  
    END LOOP;  
  
    DBMS_OUTPUT.PUT_LINE('--- Département 75000 ---');  
    FOR v_employe IN c(75000) LOOP  
        DBMS_OUTPUT.PUT_LINE(' ' || v_employe.nom);  
    END LOOP;  
END;  
/
```

Résultats attendus :

```
--- Département 92000 ---  
  Alice  
  Carlos  
  Youcef  
--- Département 75000 ---  
  Bob  
  Diana
```

Note : Un curseur paramétré peut être appelé plusieurs fois avec des valeurs différentes, évitant ainsi de déclarer plusieurs curseurs quasi-identiques.

Exercice 3 : Package de gestion des clients

Un package PL/SQL regroupe des procédures, fonctions et variables dans une unité logique. Il se compose de deux parties : la spécification (interface publique) et le corps (implémentation). La surcharge (overloading) est supportée : deux procédures peuvent avoir le même nom si leurs signatures diffèrent.

Création de la table client

```
CREATE TABLE client (  
    idClient      NUMBER(10)      NOT NULL,  
    nomClient     VARCHAR2(50)    NOT NULL,  
    prenomClient  VARCHAR2(50),  
    emailClient   VARCHAR2(100),  
    telClient     VARCHAR2(20),  
    CONSTRAINT client_pk PRIMARY KEY (idClient)  
);  
COMMIT;
```

Spécification du package

La spécification déclare deux procédures surchargées ajouterClient : l'une complète (5 paramètres), l'autre minimale (id et nom uniquement).

```
CREATE OR REPLACE PACKAGE pkg_client AS  
  
    -- Version complète : tous les attributs  
    PROCEDURE ajouterClient(  
        p_id      IN client.idClient%TYPE,  
        p_nom     IN client.nomClient%TYPE,  
        p_prenom  IN client.prenomClient%TYPE,  
        p_email   IN client.emailClient%TYPE,  
        p_tel     IN client.telClient%TYPE  
    );  
  
    -- Version minimale : id et nom seulement  
    PROCEDURE ajouterClient(  
        p_id IN client.idClient%TYPE,  
        p_nom IN client.nomClient%TYPE  
    );  
  
END pkg_client;  
/
```

Corps du package (implémentation)

Le corps implémente les deux procédures avec gestion complète des exceptions : violation de clé primaire, données invalides, et toute erreur inattendue.

```
CREATE OR REPLACE PACKAGE BODY pkg_client AS  
  
    -- Implémentation complète  
    PROCEDURE ajouterClient(  
        p_id      IN client.idClient%TYPE,  
        p_nom     IN client.nomClient%TYPE,  
        p_prenom  IN client.prenomClient%TYPE,  
        p_email   IN client.emailClient%TYPE,  
        p_tel     IN client.telClient%TYPE  
    ) IS  
    BEGIN  
        IF p_id IS NULL THEN  
            RAISE APPLICATION_ERROR(-20001, 'L''id client ne peut pas être  
NULL.');
```

```

        END IF;
        IF p_nom IS NULL OR TRIM(p_nom) = '' THEN
            RAISE APPLICATION_ERROR(-20002, 'Le nom du client est
obligatoire.');
```

```

        END IF;
        INSERT INTO client (idClient, nomClient, prenomClient, emailClient,
telClient)
        VALUES (p_id, p_nom, p_prenom, p_email, p_tel);
        COMMIT;
        DBMS_OUTPUT.PUT_LINE('Client ' || p_nom || ' ajouté (id=' || p_id ||
')');
    EXCEPTION
        WHEN DUP_VAL_ON_INDEX THEN
            ROLLBACK;
            DBMS_OUTPUT.PUT_LINE('ERREUR : id ' || p_id || ' déjà
existant.');
```

```

        WHEN VALUE_ERROR THEN
            ROLLBACK;
            DBMS_OUTPUT.PUT_LINE('ERREUR : valeur invalide ou trop
longue.');
```

```

        WHEN OTHERS THEN
            ROLLBACK;
            DBMS_OUTPUT.PUT_LINE('ERREUR inattendue : ' || SQLERRM);
    END ajouterClient;

-- Implémentation minimale
PROCEDURE ajouterClient(
    p_id IN client.idClient%TYPE,
    p_nom IN client.nomClient%TYPE
) IS
BEGIN
    IF p_id IS NULL THEN
        RAISE_APPLICATION_ERROR(-20001, 'L''id client ne peut pas être
NULL.');
```

```

    END IF;
    IF p_nom IS NULL OR TRIM(p_nom) = '' THEN
        RAISE_APPLICATION_ERROR(-20002, 'Le nom du client est
obligatoire.');
```

```

    END IF;
    INSERT INTO client (idClient, nomClient) VALUES (p_id, p_nom);
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Client ' || p_nom || ' ajouté (minimal, id='
|| p_id || ').');
```

```

    EXCEPTION
        WHEN DUP_VAL_ON_INDEX THEN
            ROLLBACK;
            DBMS_OUTPUT.PUT_LINE('ERREUR : id ' || p_id || ' déjà
existant.');
```

```

        WHEN VALUE_ERROR THEN
            ROLLBACK;
            DBMS_OUTPUT.PUT_LINE('ERREUR : valeur invalide ou trop
longue.');
```

```

        WHEN OTHERS THEN
            ROLLBACK;
            DBMS_OUTPUT.PUT_LINE('ERREUR inattendue : ' || SQLERRM);
    END ajouterClient;

END pkg_client;
/
```

Tests du package

```
SET SERVEROUTPUT ON;
```



```

-- Test 1 : insertion complète valide
EXEC pkg_client.ajouterClient(1, 'Dupont', 'Jean', 'jean@mail.com',
'0601020304');

-- Test 2 : insertion minimale valide
EXEC pkg_client.ajouterClient(2, 'Martin');

-- Test 3 : doublon de clé primaire
EXEC pkg_client.ajouterClient(1, 'Durand', 'Pierre', 'pierre@mail.com',
'0600000000');

-- Test 4 : nom NULL
EXEC pkg_client.ajouterClient(3, NULL);

-- Vérification
SELECT * FROM client;

```

Résultats attendus :

```

Client Dupont ajouté (id=1).
Client Martin ajouté (minimal, id=2).
ERREUR : id 1 déjà existant.
ERREUR : Le nom du client est obligatoire.

```

Tableau récapitulatif des exceptions

Exception	Déclencheur	Action
DUP_VAL_ON_INDEX	Clé primaire déjà existante	ROLLBACK + message erreur
VALUE_ERROR	Valeur trop longue / type incompatible	ROLLBACK + message erreur
RAISE_APPLICATION_ERROR	Validation métier (NULL, vide)	Arrêt avec code -200xx
OTHERS + SQLERRM	Toute erreur non prévue	ROLLBACK + affichage erreur Oracle

Note : SQLERRM retourne le message d'erreur Oracle de l'exception courante. SQLCODE retourne son code numérique. Très utiles dans le handler WHEN OTHERS pour diagnostiquer les erreurs inattendues.

Conclusion

Ce TP a permis d'explorer les fondamentaux du PL/SQL Oracle à travers trois axes :

- Exercice 1 : blocs anonymes, boucles, récursivité et stockage de résultats en base.
- Exercice 2 : curseurs explicites (LOOP et FOR IN), attributs implicites (SQL%ROWCOUNT, %NOTFOUND) et curseurs paramétrés.
- Exercice 3 : conception d'un package complet avec surcharge de procédures et gestion exhaustive des exceptions.

Le PL/SQL se distingue du SQL pur par sa capacité à encapsuler de la logique métier côté serveur, améliorant les performances (moins d'échanges réseau) et la maintenabilité du code.